⏱ DAY 3: COMPLETE 6-HOUR MASTER TRAINER PLAYBOOK

Deep Learning on HPC with TensorFlow, Slurm & GPUs

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Daily Duration:** 6 Full Hours (Session 1: Morning 3h + Session 2: Afternoon 3h)
Official Syllabus Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)
Day 3 Complete Terminology Glossary: `00_DAY_03_MASTER_TERMINOLOGY_AND_CONCEPT_GLOSSARY.md`
Day 3 Hands-On Lab Runbook: `02_DAY_03_HANDS_ON_LAB_AND_COMMANDS.md`

🎯 1. Official Syllabus Scope (Page 2 of PDF — Strictly Preserved)

The official curriculum specifies **strictly 6 core topics for Day 3**:

1. **ML workflows on HPC systems, Optimized libraries**
2. **Introduction to hardware-aware ML optimization**
3. **Intel-optimized TensorFlow (tensorflow-mkl, oneDNN)**
4. **Running DL models on Intel GPUs (Xeon, Habana Gaudi)**
5. **Slurm job scheduling for Deep Learning (batch jobs, resource allocation)**
6. **Training TensorFlow with Horovod & Distributed training**

> 📌 **IMPORTANT**
> **Curriculum Boundary Discipline:** Detailed microarchitecture performance profiling (Intel VTune, TMAM, Roofline Model) belongs strictly to Day 4. Generative AI, Large Language Models, and OpenVINO quantization belong strictly to Day 5. Keep Day 3 tightly focused on deep neural networks, hardware operator fusion, supercomputer accelerators, Slurm batch scheduling, and Horovod distributed gradient synchronization.

⏰ 2. Complete 6-Hour Dual-Session Master Daily Schedule

DAY 3 MASTER 6-HOUR SCHEDULE (09:30 AM - 04:30 PM)	
👷 SESSION 1: MORNING THEORY & ARCHITECTURE MASTERCLASS (09:30 AM - 12:30 PM | 3 Hours) "Deep Learning Silicon Physics → oneDNN Operator Fusion → Slurm & Horovod Math"	
09:30 - 10:15 (45m)	PART 1: Hardware-Aware DL & Intel oneDNN Primitives (Operator Fusion, AMX Matrix Engines, tensorflow-mkl)
10:15 - 11:00 (45m)	PART 2: Accelerators in HPC: Intel GPU Max & Habana Gaudi 2/3 (MME Systolic Arrays, TPCs, Integrated 24x 100GbE RoCE)
11:00 - 11:10 (10m)	☕ REJUVENATION BREAK (10 Minutes)
11:10 - 11:45 (35m)	PART 3: Supercomputer Orchestration with Slurm Workload Manager (slurmctld vs slurmd, #SBATCH Directives, Resource Binding)
11:45 - 12:15 (30m)	PART 4: Distributed Deep Learning with Horovod (Ring-AllReduce vs Parameter Server, hvd.init(), Scaling)
12:15 - 12:30 (15m)	PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser
🍱 LUNCH & INTERMISSION BREAK (12:30 PM - 01:30 PM | 60 Minutes) Cluster partition state verification, TensorFlow oneDNN environment pre-warming	
💻 SESSION 2: AFTERNOON HANDS-ON SUPERCOMPUTER LAB (01:30 PM - 04:30 PM | 3 Hours) "oneDNN Fusion Benchmarks → Slurm #SBATCH Scripts → Horovod Ring Training → 25 MCQs"	
01:30 - 02:15 (45m)	STAGE 0 & 1: Intel-Optimized TensorFlow Setup & oneDNN Fusion (Verifying TF_ENABLE_ONEDNN_OPTS=1, Fused Conv+ReLU test)
02:15 - 03:00 (45m)	STAGE 2 & 3: Slurm Batch Scripting Drill & Partition Allocations (Writing #SBATCH files, testing sbatch, squeue, slurmd)
03:00 - 03:45 (45m)	STAGE 4: Discrete-Event Slurm Simulation & GPU Resource Binding (Simulating cluster partition scheduling & queue latency)
03:45 - 04:15 (30m)	STAGE 5: Horovod Distributed Training Simulation (Ring-AllReduce) (Simulating 4-worker gradient sync, linear images/sec)
04:15 - 04:30 (15m)	ASSESSMENT & WRAP-UP: 25 Evaluated MCQs & Checklist Sign-Off (22-Point Mastery Checklist, 25 MCQs, Day 4 Preview)

🎙️ 3. Line-by-Line Minute-by-Minute Trainer Delivery Script (Session 1: Morning)

PART 1: Hardware-Aware DL & Intel oneDNN (09:30 - 10:15 | 45 Mins)

- **Slide Alignment:** Day 3 Slide Deck (Slides 1–5: Deep Learning on HPC & oneDNN).
- **Concept:** Moving from traditional ML to Deep Learning; hardware operator fusion.
- **Trainer Script (Hinglish/English):**

> "Welcome to Day 3 of the HPC Masterclass! Yesterday we saw how Scikit-learn accelerates traditional machine learning. Today, we step into the massive scale of **Deep Learning on HPC clusters**. When training a deep convolutional neural network or a Transformer, millions of matrix multiplications occur every second. If you run generic TensorFlow, every layer writes its output back to DRAM, and the next layer reads it back over the memory bus. This creates a massive **Memory Wall bottleneck**. Intel created **oneDNN (oneAPI Deep Neural Network Library)** to eliminate this! Look at **Operator Fusion**: instead of doing Convolution → Write to RAM → Read from RAM → ReLU, oneDNN combines them into a single fused hardware kernel! The ReLU activation executes while the intermediate activations are still hot in CPU L1 cache. On modern 4th/5th Gen Intel Xeon CPUs, oneDNN activates **Intel AMX (Advanced Matrix Extensions)**—specialized silicon tile registers (`tmm0` – `tmm7`) that compute 1024 BF16 operations in a single clock cycle!"

- **Whiteboard Sketch 1:**

```
Unfused Standard Flow (Chokes Memory Bus):
[Conv2D] ──► Write to DRAM (80 GB/s) ──► [BatchNorm] ──► Write to DRAM ──► [ReLU]

oneDNN Fused Kernel (Zero RAM Traffic):
+---------------------------------------------------+
|      FUSED ONEDNN KERNEL IN CPU L1 CACHE (2 TB/s)  |
|             [Conv2D + BatchNorm + ReLU]            |
+---------------------------------------------------+
```

PART 2: Supercomputer Accelerators: Intel GPU Max & Habana Gaudi (10:15 - 11:00 | 45 Mins)

- **Slide Alignment:** Day 3 Slide Deck (Slides 6–10: Modern Supercomputing Accelerators).
- **Concept:** Specialized silicon engines: Systolic Arrays, TPCs, and on-die networking.
- **Trainer Script:**

> "Why do supercomputers use accelerators alongside CPUs? Look at the **Intel Data Center GPU Max (Ponte Vecchio)**: it packs 128 Xe-cores with dedicated matrix engines called XMX (Xe Matrix eXtensions), executing dense matrix multiplications with massive FP64/FP32 throughput. Now look at **Habana Gaudi 2 and Gaudi 3**: Gaudi is not a graphics card adapted for AI. It was built from the silicon up purely for Deep Learning and Transformers!
>
> - **Matrix Math Engine (MME):** Massive hardware systolic arrays for GEMM (General Matrix Multiply).
> - **Tensor Processor Cores (TPCs):** Custom VLIW SIMD processors optimized for custom non-linear activations.
> - **The Killer Secret Weapon:** Gaudi integrates **24 ports of 100 Gigabit Ethernet (RoCE v2) directly on the silicon die!** In standard clusters, you need expensive external PCIe network cards. Gaudi communicates directly chip-to-chip over standard Ethernet cables!"

☕ BREAK (11:00 - 11:10 | 10 Minutes)

PART 3: Supercomputer Orchestration with Slurm Workload Manager (11:10 - 11:45 | 35 Mins)

- **Slide Alignment:** Day 3 Slide Deck (Slides 11–15: Slurm Architecture & Job Control).
- **Concept:** Managing hundreds of shared compute nodes without conflicts.
- **Trainer Script:**

> "In an enterprise cluster with 500 servers and 60 researchers, what happens if everyone runs `python train.py` simultaneously? The servers would crash with Out-Of-Memory errors! This is why all supercomputers use a **Batch Job Scheduler**, and the undisputed king is **Slurm (Simple Linux Utility for Resource Management)**. How Slurm works:
>
> - `slurmctld` : The central controller daemon on the head node. It manages queues, partitions, and priority.
> - `slurmd` : The worker daemon running on each compute node. It monitors resources and launches assigned jobs.
> - `sbatch` : Submits a batch script to the queue. You don't sit waiting; Slurm runs your job when resources become free!
> - `squeue` : Shows running and queued jobs.
> - `scancel` : Kills a job. In our lab today, you will write real `#SBATCH` scripts with core count, memory, and time limits!"

PART 4: Distributed Deep Learning with Horovod (11:45 - 12:15 | 30 Mins)

- **Slide Alignment:** Day 3 Slide Deck (Slides 16–20: Distributed Training & Horovod).
- **Concept:** Scaling across multiple servers using Ring-AllReduce.
- **Trainer Script:**

> "How do we train a neural network across 8 servers simultaneously?
>
> 1. **Data Parallelism:** Every server holds an identical copy of the model weights, but receives a unique slice of the dataset.
> 2. Each worker computes forward pass and backward pass on its local batch to get local gradients.
> 3. Now, workers must average their gradients before updating weights. If we used a central Parameter Server, with 64 workers, the parameter server's network card gets overwhelmed! That is why we use **Ring-AllReduce via Horovod**: Workers form a logical ring. Each rank sends partial gradients to its successor and receives from its predecessor. In exactly $2 \times (N - 1)$ transfers, every single worker gets the globally averaged gradients with **zero central bottleneck**!"

PART 5: Morning Review, 5 Rapid Check Questions & Lunch Teaser (12:15 - 12:30 | 15 Mins)

[12:15 - 12:25] 5-Question Quick Classroom Check

1. *Q1: What is Operator Fusion in Intel oneDNN and what performance bottleneck does it solve?*
   Answer: Fusing consecutive operations (Conv + Bias + ReLU) into a single hardware kernel; eliminates redundant DRAM memory read/write roundtrips.
2. *Q2: What is the specialized silicon feature on Intel Xeon Sapphire Rapids processors that accelerates matrix math?*
   Answer: Intel AMX (Advanced Matrix Extensions) with 8 hardware tile registers (`tmm0` – `tmm7`).
3. *Q3: What unique networking advantage is integrated directly on the Habana Gaudi silicon die?*
   Answer: 24 integrated ports of 100GbE RoCE v2 directly on the chip, eliminating external PCIe NIC bottlenecks.
4. *Q4: What is the difference between `slurmctld` and `slurmd`?*
   Answer: `slurmctld` is the central controller daemon running on the head node; `slurmd` is the execution daemon on each compute node.

5. *Q5: In Horovod Ring-AllReduce with N workers and gradient size S, how much data does each worker send in total?*
   Answer: $2 \cdot \frac{N-1}{N} \cdot S$ bytes, which remains nearly constant even as cluster size N scales to thousands!

[12:25 - 12:30] Lunch Break Announcement & Afternoon Lab Setup

- **What Trainer Shivam Says:**

> "Magnificent job this morning! We understand the entire deep learning stack: oneDNN operator fusion, Gaudi silicon, Slurm batch queues, and Horovod ring synchronization. We now take our 1-hour lunch break until 01:30 PM. In Session 2, we execute live deep learning labs: measuring oneDNN speedup, writing `#SBATCH` cluster scripts, and running multi-worker Horovod distributed training simulations. See you at 01:30 PM sharp!"

🍱 INTERMISSION: Lunch & Cluster Preflight (12:30 PM - 01:30 PM | 60 Mins)

- **Trainer System Checklist during break:**
  1. Verify TensorFlow oneDNN optimization flags (`export TF_ENABLE_ONEDNN_OPTS=1`).
  2. Test local Slurm emulation tools and cluster partition availability.
  3. Confirm Horovod / MPI communication sockets are open between worker ranks.
  4. Project Stage 0 preflight verification on the main lecture screen.

💻 4. SESSION 2: Afternoon Hands-On Supercomputer Lab Playbook (01:30 PM - 04:30 PM | 3 Hours)

Reference Runbook: `02_DAY_03_HANDS_ON_LAB_AND_COMMANDS.md`
Notebook Reference: `Day_03_Deep_Learning_TensorFlow_oneDNN_Slurm_Horovod.ipynb`

```
+-------------------------------------------------------------------------------------+
|                    SESSION 2: 3-HOUR HANDS-ON LAB EXECUTION PHASES                  |
+-------------------------------------------------------------------------------------+
| 01:30 - 02:15 (45m) | Phase 1: Intel-Optimized TensorFlow & oneDNN Verification (Stages 0-1) |
| 02:15 - 03:00 (45m) | Phase 2: Slurm Batch Scripting Drill & Partition Allocations (Stage 2)  |
| 03:00 - 03:45 (45m) | Phase 3: Discrete-Event Slurm Simulation & GPU Resource Binding (Stage 3)|
| 03:45 - 04:15 (30m) | Phase 4: Horovod Distributed Training Simulation (Stages 4-5)       |
| 04:15 - 04:30 (15m) | Phase 5: Student 25 MCQs Assessment & 22-Point Checklist Sign-Off   |
+-------------------------------------------------------------------------------------+
```

[01:30 - 02:15] Phase 1: Intel-Optimized TensorFlow & oneDNN Verification (Stages 0 & 1)

- **Trainer Instruction (To Class):**

> "Welcome back! Open your terminal or the Day 3 Google Colab notebook. Let's start with Stage 0 and Stage 1: We verify that our TensorFlow environment activates Intel oneDNN. Run cell 2 or check the environment: `python3 -c 'import tensorflow as tf; print(tf.__version__)'`. Notice the startup banner: `This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN)`. Now run the benchmark comparing stock convolution vs fused convolution (Conv2D + BatchNorm + ReLU). Look at the latency: operator fusion reduces runtime by **2.2x to 3.5x** because memory roundtrips are eliminated!"

[02:15 - 03:00] Phase 2: Slurm Batch Scripting Drill & Partition Allocations (Stage 2)

- **Trainer Instruction (To Class):**

> "Now look at Stage 2: We are authoring real production `#SBATCH` scripts. Write `slurm_batch_template.sh`:
>
> ```
> #!/bin/bash
> #SBATCH --job-name=tf_deep_learning
> #SBATCH --nodes=2
> #SBATCH --ntasks-per-node=4
> #SBATCH --cpus-per-task=8
> #SBATCH --time=01:00:00
> #SBATCH --partition=compute-intel
> #SBATCH --output=logs/%x_%j.out
> ```
>
> Explain every directive to the person next to you! Why `--nodes=2`? Two physical machines. Why `--ntasks-per-node=4`? Four MPI worker tasks per machine. Submit it with `sbatch slurm_batch_template.sh` and monitor with `squeue`."

[03:00 - 03:45] Phase 3: Discrete-Event Slurm Simulation & GPU Resource Binding (Stage 3)

- **Trainer Instruction (To Class):**

> "In Stage 3, we execute our discrete-event cluster scheduler simulation. Run cell 5 in the notebook. Watch the terminal: 10 simulated jobs with varying GPU, CPU, and memory demands enter the queue. The Slurm scheduler evaluates fair-share priority, checks available partitions (`gpu-hpc`, `standard-cpu`), and allocates execution slots. Look at how job dependencies and node reservations prevent resource contention!"

[03:45 - 04:15] Phase 4: Horovod Distributed Training Simulation (Stages 4 & 5)

- **Trainer Instruction (To Class):**

> "In Stage 4, we launch multi-worker distributed training with Horovod. Look at `horovod_tf_training.py`. Notice the 4 crucial steps:
>
> 1. `hvd.init()`: Initializes the distributed communication rank.
> 2. `tf.config.set_visible_devices(gpus[hvd.local_rank()])`: Pins rank to local accelerator.
> 3. `opt = hvd.DistributedOptimizer(opt)`: Wraps standard Adam with allreduce gradient averaging.
> 4. `hvd.callbacks.BroadcastGlobalVariablesHook(0)`: Broadcasts initial weights from Rank 0 to all workers! Run the 4-worker simulation and watch the images/second throughput scale linearly!"

[04:15 - 04:30] Phase 5: Student 25 MCQs Assessment & Checklist Sign-Off

- **Trainer Activity:**
  1. Open `03_DAY_03_STUDENT_CHEATSHEET_AND_MCQS.md`.
  2. Project the 25 Evaluated MCQs. Have students verify their answers on oneDNN fusion, Slurm directives, and Horovod ring communications.
  3. Verify all students complete the 22-point Day 3 Mastery Checklist.
  4. **Day 4 Teaser Announcement:**

> "Outstanding work today! You mastered the deep learning pipeline on supercomputers: oneDNN operator fusion, Habana Gaudi, Slurm job management, and Horovod distributed training. Tomorrow on Day 4, we tackle the crown jewel of supercomputing: **Performance Profiling & Optimization**! We will use Intel VTune Profiler, analyze hardware PMU counters, diagnose cache misses, explore the Roofline Model, and learn how to make slow code run at hardware limits. Be here tomorrow morning at 09:30 AM sharp!"

🛠️ 5. Top 5 Student Gotchas & 30-Second Rapid Fixes

Issue Encountered	Why It Happened	30-Second Trainer Fix
`TF_ENABLE_ONEDNN_OPTS=0` warning	oneDNN disabled by environment variable	Run `export TF_ENABLE_ONEDNN_OPTS=1` before starting Python
`sbatch: command not found`	On workstation/Colab without native Slurm installed	Run the included Python Slurm discrete-event simulator script
Horovod training deadlocks at Epoch 1	Rank 0 did not broadcast initial weights	Add `hvd.callbacks.BroadcastGlobalVariablesHook(0)` to training loop
Out of Memory (OOM) on GPU/RAM	Batch size too large for local device memory	Halve the local per-worker batch size and double the Horovod worker count
Slurm job stays in `PENDING (ReqNodeNotAvail)`	Requested more cores/GPUs than partition possesses	Check partition limits with `sinfo` and lower `--cpus-per-task`